Core Deep Learning

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Emmanuel Djegou, PhD
emmanueldjegou5@gmail.com

Course Outline

Module 3
Why Deep Learning?

1. Limits of linear models
2. Curse of dimensionality
3. Smoothness prior & its limits
4. Manifold hypothesis
5. Example: Learning XOR

Module 4
Neural Networks

6. Feedforward networks
7. Cost functions
8. Output units
9. Activation functions
10. Universal approximation
11. Architecture design

Module 5
Backpropagation

12. Gradient descent & SGD
13. Building ML algorithms
14. Computational graphs
15. Chain rule
16. Backpropagation
17. MLP training walkthrough

Linear Models: Strengths and Their Fatal Limit

Why linear models are attractive:

- Train efficiently — closed form or convex optimisation
- Theoretical guarantees: convex loss $\Rightarrow$ global optimum
- Interpretable: each weight is a direct effect size
- Well-studied and reliable

The fundamental limitation:
A linear model $f(x) = w^\top x + b$ produces an output that is a linear function of the input.
It **cannot** capture any interaction between input features.

Why interactions matter

Suppose we want to predict whether an image contains a face. Whether pixel 3 is bright matters *only in relation to* pixel 7. A linear model applies a fixed weight to each pixel independently — it has no way to say "pixel 3 is relevant only when pixel 7 is also bright."

Consequence for deep learning

Deep networks overcome this by first transforming the input into a new representation where a linear model *can* solve the problem. This is the core idea.

Three Options for Nonlinearity

We want to apply a linear model not to x directly but to a transformed input $\phi(x)$, where ϕ captures nonlinear structure.

Option 1: Generic ϕ

Use a very high-dimensional feature map (e.g. RBF kernel: infinite-dimensional ϕ).

Problem: uses only the smoothness prior. Cannot encode task-specific structure. Poor generalisation on hard tasks.

Option 2: Manual ϕ

Human experts design features by hand for each domain (speech, vision, text, ...).

Problem: requires decades of effort per task. Little transfer between domains. Does not scale.

Option 3: Learn ϕ

The deep learning approach. Learn the feature transformation from data:

$$y = \phi(x; \theta)^\top w$$

θ parametrises the representation; w maps it to output.

Gives up convexity, but the benefits outweigh the cost.

Option 3 is the only approach that can automatically discover representations suited to any given task.

Curse of Dimensionality: Why Learning Breaks

Core idea (Bellman, 1957)

The number of possible configurations grows **exponentially** with the number of dimensions.

Key intuition: Even a simple notion like "coverage of space" becomes impossible in high dimensions.

- In low dimensions, data can cover the space
- In high dimensions, almost all space is empty
- Learning requires seeing representative examples

Main consequence

Most learning problems become impossible without structure or strong assumptions.

Curse of Dimensionality: Exponential Explosion

Discretisation argument:

Assume each input dimension is divided into v bins.

Each additional dimension multiplies the number of regions:

- 1D: v
- 2D: v^2
- 3D: v^3
- dD: v^d

Key interpretation: Each new feature dimension creates a full copy of the space.

Data requirement: To observe at least one sample per region:

$$O(v^d)$$

Extreme case:

$$d = 100,\ v = 2 \Rightarrow 2^{100} \approx 10^{30}$$

Key insight:

- The number of regions grows exponentially with dimension
- The number of training samples grows only linearly with data collection
- Therefore most regions in high dimensions are never observed

Conclusion:
Naive learning methods fail in high dimensions without additional structure or assumptions.

Curse of Dimensionality: What It Means for ML

How traditional algorithms cope: they assume new points should look like their nearest training example.

In low dimensions, this works:

- Most grid cells are occupied
- The nearest training point is close
- Local interpolation gives good predictions

In high dimensions, this breaks:

- Most cells are empty
- The nearest training point may be far away
- Local interpolation extrapolates wildly
- The model has never seen anything like the test point

Real-world example: image classification

- A 64×64 grayscale image: $d = 4{,}096$
- With 8-bit pixels ($v = 256$): 256^{4096} possible images
- Training sets of millions of images: still a tiny fraction of the space

The core challenge for deep learning

How can we generalise to new configurations when nearly every test point is unlike anything we have seen? We need to exploit **structure** in the data — not just local proximity.

The Smoothness Prior: Definition

Local constancy (smoothness) prior

The function we want to learn should not change much within a small neighbourhood:

$$f^*(x) \approx f^*(x + \epsilon) \quad \text{for small } \epsilon$$

What this prior says:

- If x is a labelled training example, then points *near* x probably have the same label
- If we know the output at several nearby points, we can average or interpolate them
- Generalisation is local: each training example informs prediction in its surrounding neighbourhood only

Algorithms that rely exclusively on this prior:

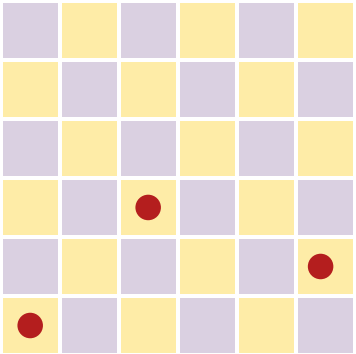
- k-Nearest Neighbours
- Kernel machines with local kernels
- Decision trees
- Naïve Bayes with local density estimates

Works well when the function is smooth and the dimension is low. The smoothness assumption is *not wrong*, it is just *insufficient* for high-dimensional AI tasks.

Why the Smoothness Prior Fails at Scale

- **Core limitation of local methods**
  - Each training example only affects a local neighbourhood
  - To learn a new region, we need data inside that region
- **Scaling consequence**
  - To distinguish $O(k)$ regions requires $O(k)$ examples
  - k-NN and kernel methods grow with the number of regions, not structure
- **High-dimensional explosion**
  - Number of regions grows exponentially with dimension d
  - Local methods therefore require exponentially many examples
- **Key takeaway**
  - k-NN with m samples can represent at most m effective regions

Intuition: The Checkerboard Problem



Only 3 training examples, most regions unknown

- **Local learning behaviour**
  - Each example labels only its nearby region
  - No information transfers to distant regions
- **Failure mode**
  - Unobserved squares must be guessed
  - Correct solution requires one example per region
- **High-dimensional implication**
  - d binary features create 2^d regions
  - Local methods scale with regions, not structure

The Key Insight: Non-Local Generalisation

Is there hope? Yes, if we introduce *dependencies between regions*.

Key result

$O(2^k)$ regions can be represented using only $O(k)$ examples, provided we assume structure linking regions together.

How deep learning achieves this:

- Learns shared features across many regions
- A single feature (for example, "has an eye") applies to many image patches
- k features can represent 2^k combinations
- Each example informs all regions that share its features

Why Deep Learning Scales: Distributed Representations

Local vs distributed learning

Local	**Distributed (deep)**
Each example affects one region	Each example updates many features
$O(k)$ samples $\rightarrow O(k)$ regions	$O(k)$ samples $\rightarrow O(2^k)$ regions
kNN, kernels, trees	Neural networks

Deep learning generalises by sharing features across exponentially many regions, which local methods cannot do.

The Manifold Hypothesis: Definition

Manifold

A **manifold** is a set of points that locally resembles a Euclidean space of lower dimension than the ambient space. Each point has a neighbourhood that looks like a small flat patch.

Everyday example:

- The surface of the Earth: 2D manifold embedded in 3D space
- Locally it looks flat (a 2D plane)
- Globally it is curved (a sphere)

In machine learning: we say data lies near a *low-dimensional manifold embedded in a high-dimensional space*.

The intrinsic degrees of freedom of the data are much fewer than the ambient dimension.

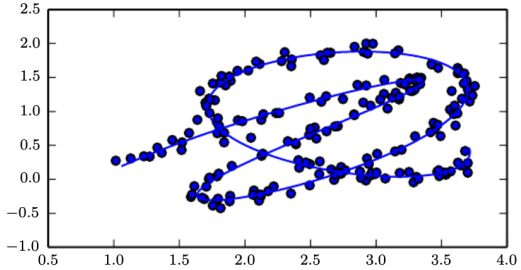


Figure: Data sampled in 2D but concentrated near a 1D manifold (solid line shows the underlying structure to be learned).

The Manifold Hypothesis: Evidence

Why believe data lies on low-dimensional manifolds? Two independent arguments:

Argument 1: Probability is concentrated

- Sample a 256×256 image by picking each pixel uniformly at random: the result looks like TV static
- Real face images, text images, or speech signals essentially never arise from such random sampling
- The probability mass of natural data occupies a *negligibly small fraction* of the total image/audio/text space

This means: interesting inputs cluster together, they do not spread uniformly through $\mathbb{R}^n$.

Argument 2: Inputs are connected

- We can imagine smooth transformations that stay within the set of valid inputs:
  - Gradually dim or brighten an image
  - Slowly rotate an object
  - Shift face pose left or right
  - Gradually change a word in a sentence
- These transformations trace a *path along the manifold*
- Valid inputs are connected to other valid inputs by such paths

Implication: represent data in *manifold coordinates*, not in $\mathbb{R}^n$. Deep learning learns these coordinates.

Example: Learning XOR — The Task

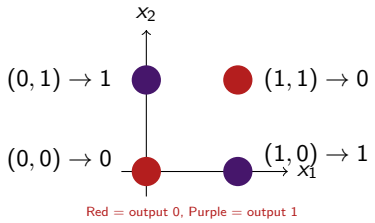
The XOR function:

XOR ("exclusive or") on two binary inputs:

x_1	x_2	$y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Goal: learn a function $f(x;\theta)$ that matches this table exactly on all four inputs. No test set: just fit the four training points.

Visualising the problem:



Notice: the two red points (output 0) are at diagonally opposite corners. No straight line can separate the red from the purple points.

Example: Learning XOR — Why a Linear Model Fails

Attempt: fit a linear model $\hat{y} = \mathbf{w}^\top x + b$ using the mean squared error loss.

Solve the normal equations:

The four training examples give:

$$J(\mathbf{w}, b) = \frac{1}{4} \sum_{i=1}^{4} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Solving $\nabla_{\mathbf{w},b} J = 0$ gives:

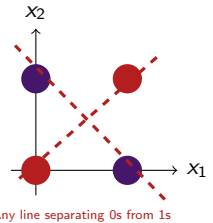
$$w_1 = 0, \quad w_2 = 0, \quad b = \tfrac{1}{2}$$

The model outputs $\hat{y} = 0.5$ for *every* input.

Why? When $x_1 = 0$: output must increase with x_2 (from 0 to 1). When $x_1 = 1$: output must *decrease* with x_2 (from 1 to 0).

A linear model applies a fixed coefficient w_2 to x_2. It cannot change w_2 based on the value of x_1.

Geometric view:



Conclusion: XOR is *not linearly separable*. A linear model is fundamentally unable to represent this function.

Example: Learning XOR with a Neural Network

Problem: XOR is not linearly separable, so linear models fail.

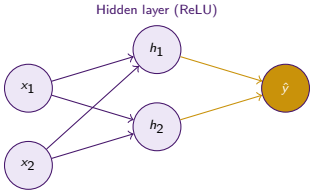
Solution: A feedforward network with one hidden layer (ReLU)

$$f(x) = w^\top \max\{0,\, Wx + c\} + b$$

Closed-form parameters:

$$W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad c = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

$$w = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \quad b = 0$$



Key idea: The hidden layer creates a representation where XOR becomes linearly separable.

Example: Learning XOR (Verification)

Forward pass:

$$z = Wx + c, \quad h = \max(0, z), \quad \hat{y} = w^\top h$$

x_1	x_2	z_1	z_2	h_1	h_2	$\hat{y}$
0	0	0	-1	0	0	0
0	1	1	0	1	0	1
1	0	1	0	1	0	1
1	1	2	1	2	1	0

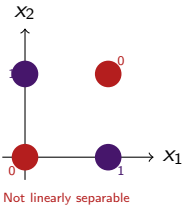
Result: The network solves XOR with zero training error.

Key insight: A hidden layer learns a representation that makes the problem linearly separable.

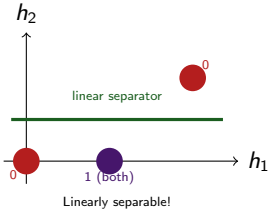
Example: Learning XOR — Representation Learning

What did the hidden layer actually do?

Original input space:



Learned feature space (after ReLU):



- Inputs $(0,1)$ and $(1,0)$ — both output 1 — are mapped to the **same point** $h = (1,0)$ in feature space
- Input $(0,0)$ maps to $(0,0)$ and $(1,1)$ maps to $(2,1)$
- In the new space, a linear model *can* separate the two classes

The network solved XOR by learning a new representation in which the problem became easy.
This is representation learning.

Deep Feedforward Networks: Definition

Definition

A **deep feedforward network** (MLP) is a function composed of layers:

$$f(x) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(x)))$$

Key property: Information flows only forward from input to output.

- No feedback connections
- No cycles in computation
- Output is a composition of functions

Terminology:

- L: depth of the network
- $f^{(1)}, \dots, f^{(L-1)}$: hidden layers
- $f^{(L)}$: output layer

Deep Feedforward Networks: Interpretation

Why "hidden"?

Training data specifies only input–output pairs (x, y).

It does **not specify intermediate representations**.

The learning algorithm must discover useful hidden representations automatically.

Why "neural"?

Each unit computes:

$$h_i = g(w_i^\top x + b_i)$$

- Receives multiple inputs
- Produces a scalar activation
- Inspired by biological neurons, but not biologically accurate

Key idea:

- Hidden layers learn representations
- Output layer maps representation to prediction
- Learning finds both representation and mapping

Core perspective:

Deep learning = representation learning + function approximation

Deep Feedforward Networks: Architecture

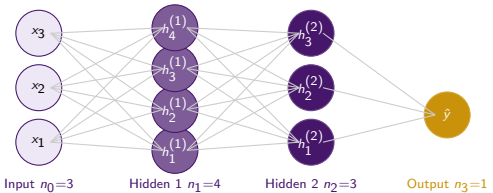
A typical chain structure:

$$h^{(k)} = g^{(k)}\big(W^{(k)}\, h^{(k-1)} + b^{(k)}\big)$$

- $W^{(k)} \in \mathbb{R}^{n_k \times n_{k-1}}$: weight matrix
- $b^{(k)} \in \mathbb{R}^{n_k}$: bias vector
- $g^{(k)}$: element-wise activation function
- n_k: **width** of layer k
- $h^{(0)} = x$, $h^{(L)} = \hat{y}$

Two key design choices:

1. How many layers? (depth)
2. How many units per layer? (width)



Depth $L = 3$. Width varies by layer. Every unit in layer k connects to every unit in layer $k+1$ (fully connected).

Cost Functions: The Maximum Likelihood Principle

The MLE recipe for neural networks

1. Define a conditional probability model $p(y|x;\theta)$
2. The cost function is the **negative log-likelihood**:

$$J(\theta) = -\mathbb{E}_{x,y\sim\hat{p}_{\text{data}}}[\log p(y|x;\theta)]$$

3. Minimising J = maximising the likelihood = making the model's predictions match the data distribution

Why this approach?

- No manual cost function design is needed; derive it from your distributional assumption
- Principled and general
- Works for any output type

Cost function and the output distribution:

Output	Cost function
Gaussian $p(y\|x)$	Mean squared error
Bernoulli $p(y\|x)$	Bin cross-entropy
Cat (softmax)	Cross-entropy

Key consequence

MSE is not an arbitrary choice. It is the NLL under a Gaussian output model. Cross-entropy is the NLL under a categorical model. The assumptions determine the cost.

Cost Functions: Why NLL and Not MSE for Classification?

The saturation problem with MSE:

Many output units involve a sigmoid or softmax, which can *saturate*: as $|z| \to \infty$, the gradient of $\sigma(z)$ approaches 0.

If we use MSE with a sigmoid output:

$$\frac{\partial}{\partial z}(\hat{y} - y)^2 = 2(\hat{y} - y) \cdot \sigma'(z)$$

When the model is confidently *wrong* (e.g. $\hat{y} \approx 1$ but $y = 0$), $\sigma'(z) \approx 0$, so the gradient is nearly zero. The model cannot correct itself.

NLL avoids this problem:

For binary classification, the NLL loss is:

$$J = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

When $\hat{y} = \sigma(z)$, the log undoes the exp inside σ. The gradient is:

$$\frac{\partial J}{\partial z} = \hat{y} - y$$

This is **always large when the model is wrong**.

Rule of thumb

Always pair sigmoid/softmax output units with cross-entropy loss. Never with MSE.

Output Units: Linear and Sigmoid

Linear units — continuous output

$$\hat{y} = W^\top h + b \qquad (\hat{y} \in \mathbb{R})$$

- Assumes $p(y|x) = \mathcal{N}(y; \hat{y}, I)$
- MLE $\Rightarrow$ minimise MSE
- No saturation: gradient is constant
- Used for regression tasks

Sigmoid units — binary output

$$\hat{y} = \sigma(w^\top h + b) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

- Assumes $p(y|x) = \text{Bernoulli}(\hat{y})$
- MLE $\Rightarrow$ minimise binary cross-entropy
- Thresholded linear units fail because gradient is zero outside $[0, 1]$
- Sigmoid ensures a non-zero gradient whenever the model has the wrong answer
- Used for binary classification

Output unit type determines the cost function. Linear $\leftrightarrow$ MSE. Sigmoid $\leftrightarrow$ binary cross-entropy.

Output Units: Softmax

Softmax units — k-class output

Given a vector of pre-activations $z \in \mathbb{R}^k$:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

- All outputs in $(0, 1)$ and sum to 1: valid probability distribution
- Generalises sigmoid (sigmoid = softmax for $k = 2$)
- MLE $\Rightarrow$ minimise categorical cross-entropy
- Numerically stable: compute $\text{softmax}(z - \max_i z_i)$

Why NLL works well with softmax:

$$\log \text{softmax}(z)_y = z_y - \log \sum_j e^{z_j}$$

- The first term z_y always contributes directly to the gradient — it never saturates
- The second term $\log \sum_j e^{z_j} \approx \max_j z_j$ penalises the most active incorrect prediction
- Learning always has a clear signal

Softmax with squared error fails

Squared error loss does not undo the saturation of the softmax. Gradients vanish when the model is confidently wrong — the worst possible time.

Activation Functions: Why We Need Them

What if we used no activation function?

Consider a two-layer network:

$$f(x) = W^{(2)}\left(W^{(1)}x + b^{(1)}\right) + b^{(2)}$$

This simplifies to:

$$f(x) = \underbrace{W^{(2)}W^{(1)}}_{\text{one matrix}}x + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{\text{one vector}}$$

A stack of linear layers is still a linear function. Adding more layers without nonlinearities gains nothing.

The role of the activation function:

We apply a **fixed nonlinear function** g after each affine transformation:

$$h = g(Wx + b)$$

This breaks the linearity and allows the network to represent nonlinear functions.

Requirements of a good activation function:

- Nonlinear (obviously)
- Provides useful gradients for learning
- Computationally inexpensive
- Allows the network to be trained stably

Activation functions are the source of nonlinearity in neural networks. Without them, depth provides no benefit.

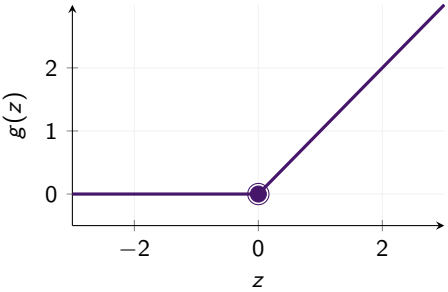
Activation Functions: ReLU

Rectified Linear Unit (ReLU)

$$g(z) = \max\{0, z\}$$

Why ReLU is the default:

- **Nearly linear:** gradient is exactly 0 (inactive) or exactly 1 (active) — no vanishing gradient when active
- **Easy to optimise:** behaves like a linear model for active units; gradient direction is clear
- **Sparse activations:** many units are zero at once; computationally efficient
- **Universal approximation:** piecewise linear functions with ReLUs can approximate any continuous function



Non-differentiability at $z = 0$:

In practice this causes no problems. The probability of landing exactly at $z = 0$ during training is essentially zero. Software returns the left or right derivative (both 0 or 1) without error.

Activation Functions: ReLU Generalisations

A drawback of ReLU: units with $z < 0$ output zero and have zero gradient — they cannot learn from those examples. Several generalisations fix this.

Leaky ReLU

$$g(z) = \max(0, z) + \alpha \min(0, z)$$

$\alpha = 0.01$ (fixed).

Gradient is α for $z < 0$: never fully zero. Helps with "dying ReLU" problem.

PReLU

$$g(z) = \max(0, z) + \alpha_i \min(0, z)$$

α_i is a *learnable* parameter per unit.

More flexible than leaky ReLU. Learns the best slope for negative inputs.

Maxout

$$g(z)_i = \max_{j \in G_i} z_j$$

Groups k linear units; outputs the max.

Can learn the activation function itself. Requires more parameters (k weight vectors per unit).

All ReLU variants share the principle: keep behaviour close to linear to make optimisation easy, while allowing nonlinearity.

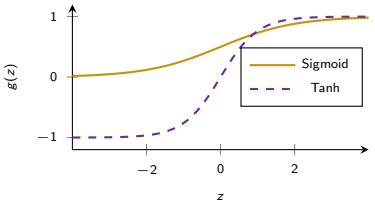
Activation Functions: Sigmoid and Tanh

Logistic sigmoid:

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Hyperbolic tangent:

$$g(z) = \tanh(z) = 2\sigma(2z) - 1 \in (-1, 1)$$



The saturation problem:

- Both functions saturate for large $|z|$: the gradient approaches zero
- During backpropagation, gradients are multiplied layer by layer. Near-zero gradients vanish rapidly (**vanishing gradient problem**)
- Learning in early layers becomes extremely slow

When are they still used?

- **Sigmoid:** output layer for binary classification (pair with cross-entropy)
- **Tanh:** preferred over sigmoid when a sigmoidal unit is needed (e.g. in RNNs), because $\tanh(0) = 0$ resembles the identity near 0, making training easier
- Both are **discouraged as hidden activations** in deep feedforward networks

Universal Approximation Theorem

Universal approximation theorem (Hornik et al., 1989; Cybenko, 1989)

A feedforward network with:

- a linear output layer,
- at least one hidden layer,
- a squashing activation function (e.g. sigmoid),
- and enough hidden units,

can approximate any Borel measurable function from one finite-dimensional space to another to any desired degree of accuracy.

(Extended to ReLUs: Leshno et al., 1993)

What it means:

- Any continuous function can be represented, in principle
- A single hidden layer is *sufficient* in terms of expressiveness

What it does NOT mean:

- It does not say the training algorithm will *find* the function
- It does not say how many units are needed (could be exponentially many)
- It does not say the network will *generalise* correctly

The catch

In the worst case, a single hidden layer may need exponentially many units, one per input configuration that needs to be distinguished.

Why Depth Helps: Exponential Efficiency

Shallow vs. deep:

- There exist functions that a depth-d network can represent with $O(n)$ units, but that a depth-$(d-1)$ network can only represent with $O(2^n)$ units
- Depth provides an *exponential* advantage in representational capacity for certain function families

Formal result (Montufar et al., 2014):

A deep ReLU network with depth l and n units per hidden layer can carve out

$$O\left(n^d \cdot k^{l-1}\right) \text{ linear regions}$$

— exponential in depth l.

Intuition: folding the input space

Each ReLU layer folds the input space along a hyperplane (the zero boundary of the ReLU). Composing l layers folds l times, creating exponentially many distinct regions.

What depth encodes as a prior:

- The target function is a *composition of simpler functions*
- Higher layers represent higher-level abstractions
- Lower layers share features across many higher-level concepts

Use deep models when you believe the target function has hierarchical structure. Almost all AI tasks do.

Architecture Design: Depth and Width

Depth (number of layers):

- Deeper networks can represent complex hierarchical functions with fewer total parameters
- Depth improves generalisation empirically — not just training fit
- Very deep networks can be harder to optimise (vanishing gradients, poor initialisation)
- Start shallow; increase depth based on validation error

Width (units per layer):

- Wider layers increase capacity within a single layer
- Less efficient than increasing depth for most AI tasks

Empirical evidence (Goodfellow et al., 2014):

- Transcribing address numbers from photos
- Increasing depth from 3 to 11 layers consistently improved accuracy
- Increasing the number of parameters by adding width to a 3-layer network did *not* match the gains from depth

Practical rule

The best architecture for a task cannot be derived from theory alone. Monitor validation error and use it to guide choices. More depth is usually worth trying before more width.

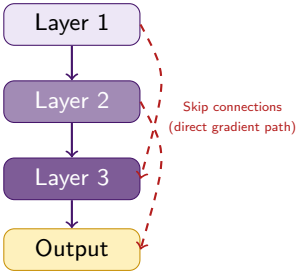
Architecture Design: Connections and Skip Connections

Fully connected layers:

- Every unit in layer k connects to every unit in layer $k+1$
- $O(n_k \cdot n_{k+1})$ parameters per layer pair
- General purpose; used in most feedforward layers

Skip connections:

- Add a direct edge from layer i to layer $i+2$ (or further)
- The gradient can flow directly from deeper layers back to earlier ones
- Helps with the *vanishing gradient* problem in very deep networks
- Used extensively in ResNets (deep vision networks)



Skip connections create shortcut paths for gradient flow. Without them, gradients through 50+ layers can vanish to nearly zero.

Gradient Descent: The Foundation

The optimisation problem:

Find θ that minimises $J(\theta)$.

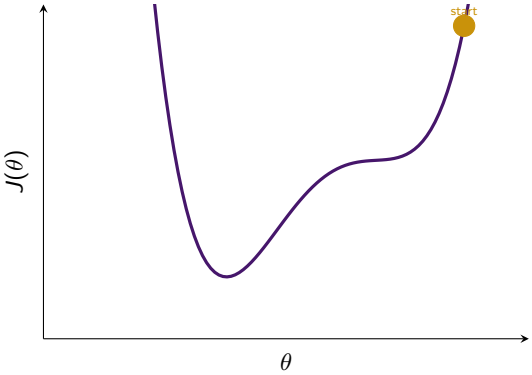
Gradient descent: iteratively move in the direction of steepest descent:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

$\epsilon > 0$ is the **learning rate** (step size).

For neural networks:

- The loss $J(\theta)$ is non-convex
- No guarantee of reaching a global minimum
- In practice, gradient descent finds solutions with very low loss quickly enough to be useful
- Proper weight initialisation is critical



Gradient descent follows the slope downhill. For non-convex surfaces, the final minimum found depends on the starting point.

Gradient Descent: The Foundation

The optimisation problem:

Find θ that minimises $J(\theta)$.

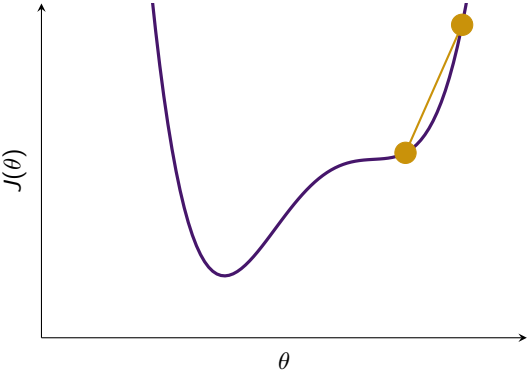
Gradient descent: iteratively move in the direction of steepest descent:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

$\epsilon > 0$ is the **learning rate** (step size).

For neural networks:

- The loss $J(\theta)$ is non-convex
- No guarantee of reaching a global minimum
- In practice, gradient descent finds solutions with very low loss quickly enough to be useful
- Proper weight initialisation is critical



Gradient descent follows the slope downhill. For non-convex surfaces, the final minimum found depends on the starting point.

Gradient Descent: The Foundation

The optimisation problem:

Find θ that minimises $J(\theta)$.

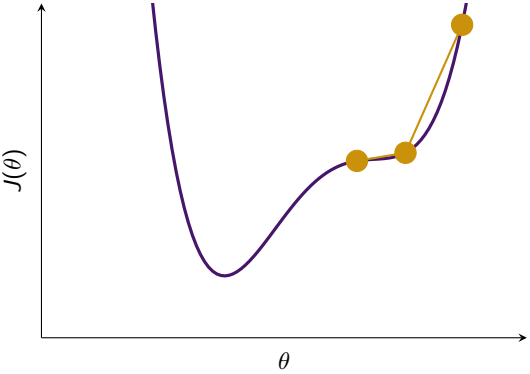
Gradient descent: iteratively move in the direction of steepest descent:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

$\epsilon > 0$ is the **learning rate** (step size).

For neural networks:

- The loss $J(\theta)$ is non-convex
- No guarantee of reaching a global minimum
- In practice, gradient descent finds solutions with very low loss quickly enough to be useful
- Proper weight initialisation is critical



Gradient descent follows the slope downhill. For non-convex surfaces, the final minimum found depends on the starting point.

Gradient Descent: The Foundation

The optimisation problem:

Find θ that minimises $J(\theta)$.

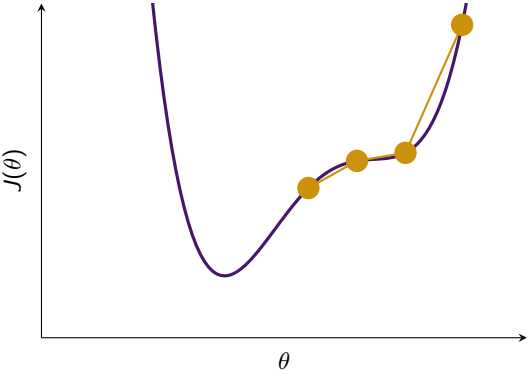
Gradient descent: iteratively move in the direction of steepest descent:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

$\epsilon > 0$ is the **learning rate** (step size).

For neural networks:

- The loss $J(\theta)$ is non-convex
- No guarantee of reaching a global minimum
- In practice, gradient descent finds solutions with very low loss quickly enough to be useful
- Proper weight initialisation is critical



Gradient descent follows the slope downhill. For non-convex surfaces, the final minimum found depends on the starting point.

Gradient Descent: The Foundation

The optimisation problem:

Find θ that minimises $J(\theta)$.

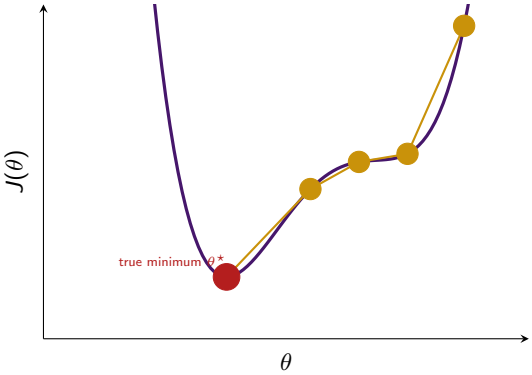
Gradient descent: iteratively move in the direction of steepest descent:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

$\epsilon > 0$ is the **learning rate** (step size).

For neural networks:

- The loss $J(\theta)$ is non-convex
- No guarantee of reaching a global minimum
- In practice, gradient descent finds solutions with very low loss quickly enough to be useful
- Proper weight initialisation is critical



Gradient descent follows the slope downhill. For non-convex surfaces, the final minimum found depends on the starting point.

Stochastic Gradient Descent

The problem with full gradient descent:

$$\nabla_\theta J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta L(x^{(i)}, y^{(i)}, \theta)$$

Computing this sum over **all** m training examples costs $O(m)$ per step. With billions of examples, this is prohibitively slow.

The key insight: the gradient is an expectation. An *expectation* can be *estimated* from a small random sample.

SGD algorithm:

1. Sample a **minibatch** $\mathcal{B}$ of m' examples uniformly from the training set ($m' \ll m$, typically 32–256)
2. Compute the gradient estimate:

$$g = \frac{1}{m'}\sum_{i=1}^{m'} \nabla_\theta L_i$$

3. Update:

$$\theta \leftarrow \theta - \epsilon\, g$$

4. Repeat

Cost per update: $O(m')$, independent of m.

SGD: Properties and Asymptotics

Why SGD works:

- The minibatch gradient g is an unbiased estimate of the true gradient $\nabla_\theta J$
- Updates are noisy, but noise can help escape sharp local minima
- For large m, the model often reaches its best test error before seeing every training example once
- Asymptotic cost in m: $O(1)$, meaning adding more data does not slow training once the model has converged

Comparison with kernel methods:

Kernel methods require an $m \times m$ Gram matrix with cost $O(m^2)$. For $m = 10^9$, this corresponds to 10^{18} operations, which is infeasible in practice.

SGD has constant cost per update in m. This is one reason deep learning scales to large datasets while kernel methods do not.

SGD is the engine behind nearly all of deep learning. Every practical improvement such as momentum, Adam, and RMSprop builds on this basic idea.

Building a Machine Learning Algorithm: The Recipe

Nearly all deep learning algorithms share this structure

Specify four components: **dataset**, **cost function**, **model**, **optimisation algorithm**.

Example: supervised MLP

- **Dataset:** labelled pairs (x, y) from $\hat{p}_{\text{data}}$
- **Model:** $p(y|x;\theta) = \text{softmax}(f(x;\theta))$
- **Cost:** $J(\theta) = -\mathbb{E}[\log p(y|x;\theta)] + \lambda\|W\|^2$
- **Optimisation:** SGD or Adam

Example: PCA (unsupervised)

- **Dataset:** unlabelled x from $\hat{p}_{\text{data}}$
- **Model:** reconstruction $r(x) = ww^\top x$, $\|w\| = 1$
- **Cost:** $J(w) = \mathbb{E}\|x - r(x;w)\|^2$
- **Optimisation:** eigenvector decomposition

Why this framing is useful:

- Different algorithms can be understood as different instantiations of the same recipe
- You can mix and match: change just the model, or just the cost, or just the optimiser
- Makes it easy to extend: want regularisation? Add a term to J. Want a different output? Change the model.

Key observation

The cost function typically includes a term for statistical estimation (NLL) and optionally regularisation terms. The model and cost together determine everything about what is learned.

Computational Graphs: Definition

Computational graph

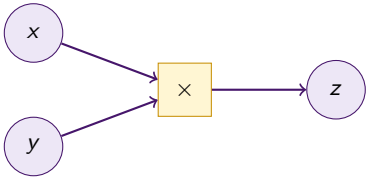
A directed acyclic graph (DAG) where:

- Each **node** represents a variable (scalar, vector, matrix, or tensor)
- Each **directed edge** $x \to y$ indicates that y was computed by applying an operation to x
- **Leaf nodes** are inputs or parameters (no incoming edges)
- The **output node** is the scalar loss J

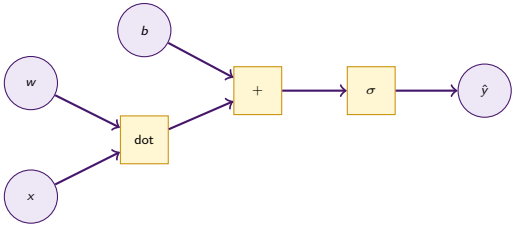
Why computational graphs?

- Make computations explicit and traceable
- Enable automatic, systematic differentiation
- The structure of the graph directly determines how backpropagation proceeds

Example: $z = x \cdot y$



Example: $\hat{y} = \sigma(w^\top x + b)$



The Chain Rule: Scalar Case

Scalar chain rule

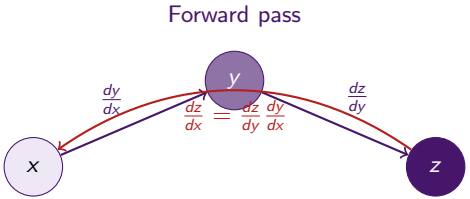
If $y = g(x)$ and $z = f(y) = f(g(x))$, then:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

Example: $x = 2$, $y = x^2 = 4$, $z = \log y = \log 4$

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} = \frac{1}{y} \cdot 2x = \frac{1}{4} \cdot 4 = 1$$

A neural network is a long chain of compositions. The chain rule lets us compute $\frac{\partial J}{\partial w}$ for any parameter w anywhere in the network, by multiplying the local derivatives along the path from w to J.



Forward pass: compute and store all intermediate values $y = g(x)$.

Backward pass: use stored values to compute the chain rule product from output back to input.

The Chain Rule: Vector and Tensor Case

Vector case:

Suppose $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$, $y = g(x)$, $z = f(y) \in \mathbb{R}$.

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i}$$

In matrix form:

$$\nabla_x z = J_g^\top \nabla_y z$$

where $J_g \in \mathbb{R}^{n \times m}$ is the Jacobian of g.

Intuition: the gradient w.r.t. x is the Jacobian (local linearisation of g) times the gradient already computed w.r.t. y.

Tensor case:

If $\mathbf{Y} = g(\mathbf{X})$ and $z = f(\mathbf{Y})$, using a single index i for all elements:

$$(\nabla_{\mathbf{X}} z)_i = \sum_j (\nabla_{\mathbf{Y}} z)_j \frac{\partial Y_j}{\partial X_i}$$

This is identical in structure to the vector case. Tensors are just multi-dimensional arrays and the chain rule is the same.

Backprop in one sentence

Backpropagation is the chain rule, applied in reverse topological order through the computational graph, storing each Jacobian-gradient product to avoid recomputation.

Backpropagation: The Idea

The naive approach (symbol chain rule expansion):

To compute $\frac{\partial J}{\partial w}$, write down every path from w to J in the computational graph and multiply the local derivatives along each path.

Problem: the number of paths grows *exponentially* with depth. Evaluating each path separately recomputes the same subexpressions over and over.

For a chain $x \to y \to z \to J$, the naive expansion of $\frac{\partial J}{\partial x}$ recomputes $\frac{\partial y}{\partial x}$ once per path through y.

Backprop's solution: dynamic programming

1. Do one forward pass; **store** all intermediate values
2. Do one backward pass; compute each node's gradient **exactly once** and store it in a table
3. To get the gradient at any parent node: look up the child's stored gradient and multiply by the local Jacobian

Result

Each node's gradient is computed once. Total cost $= O(|\text{edges}|)$, the same order as the forward pass.

Compared to the naive approach ($O(2^n)$ in the worst case), this is an exponential speedup.

Backpropagation: Forward Pass

Step 1 of backprop: the forward pass. Compute and *store* all intermediate values.

For a single-hidden-layer MLP with parameters $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$:

1. $h^{(0)} = x$ (input)
2. $a^{(1)} = W^{(1)} h^{(0)} + b^{(1)}$ (pre-activation)
3. $h^{(1)} = g(a^{(1)})$ (activation)
4. $a^{(2)} = W^{(2)} h^{(1)} + b^{(2)}$ (pre-activation)
5. $\hat{y} = h^{(2)} = g^{(2)}(a^{(2)})$ (output)
6. $J = L(\hat{y}, y) + \Omega(\theta)$ (loss)

What gets stored:

- All pre-activation values $a^{(k)}$
- All activation values $h^{(k)}$

These are needed during the backward pass.

Memory cost:

Storing all activations costs $O(L \cdot m \cdot n_{\max})$, where L = depth, m = minibatch size, $n_{\max}$ = maximum layer width.

Memory is the main cost of backprop

We must keep activations for *all* layers in memory simultaneously during the forward pass.

Backpropagation: Backward Pass

Step 2: the backward pass. Propagate gradients from the loss back through each layer.

Algorithm (for $k = L, L-1, \ldots, 1$):

1. **Initialise:** $g \leftarrow \nabla_{\hat{y}} J = \nabla_{\hat{y}} L(\hat{y}, y)$
2. **Convert to pre-activation gradient** (element-wise if g is element-wise):

$$g \leftarrow g \odot f'(a^{(k)})$$

3. **Accumulate parameter gradients:**

$$\nabla_{b^{(k)}} J = g, \quad \nabla_{W^{(k)}} J = g\, h^{(k-1)\top}$$

4. **Propagate to previous layer:**

$$g \leftarrow W^{(k)\top} g$$

Reading the equations:

- Step 2: the activation function's derivative "gates" the gradient. For ReLU: $f'(z) = \mathbf{1}[z > 0]$ — gradient passes through active units only
- Step 3: $\nabla_{W^{(k)}} J = g\, h^{(k-1)\top}$ is an outer product: each gradient update is the product of the upstream gradient and the incoming activation
- Step 4: $W^{(k)\top} g$ rotates the gradient back into the previous layer's space — this is the "propagation" step

After all layers are processed, we have gradients for every parameter. SGD uses them to update θ.

Backpropagation: Two Implementation Styles

Symbol-to-number (Torch, Caffe)

- Accept a graph and specific numeric input values
- Execute forward pass: compute numeric activations
- Execute backward pass: compute numeric gradients
- Return gradients as numbers
- Graph structure is implicit in code

Symbol-to-symbol (TensorFlow, PyTorch)

- Accept a computational graph
- Build a *new graph* describing how to compute derivatives
- Evaluate later with any specific inputs
- **Advantage:** derivatives are expressions — can differentiate again (higher-order derivatives)

Practical note: both produce identical gradients. Modern frameworks use symbol-to-symbol internally.

Higher-order derivatives: $Hv = \nabla_x\left((\nabla_x f)^\top v\right)$ — useful for second-order methods.

MLP Training: Putting It All Together

The complete training loop

Forward pass $\rightarrow$ compute loss $\rightarrow$ backward pass $\rightarrow$ parameter update $\rightarrow$ repeat

One-hidden-layer MLP:

$$H = \max\{0, XW^{(1)}\}, \quad \hat{P} = \text{softmax}(HW^{(2)})$$

$$J = \underbrace{J_{\text{MLE}}}_{\text{cross-entropy}} + \lambda\left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2\right)$$

Forward:

1. $U^{(1)} = XW^{(1)}$
2. $H = \max\{0, U^{(1)}\}$
3. $U^{(2)} = HW^{(2)}$
4. $J = \text{CE}(y, U^{(2)}) + \lambda(\cdots)$

Backward:

1. $G \leftarrow \nabla_{U^{(2)}} J_{\text{MLE}}$ (softmax-CE gradient)
2. $\nabla_{W^{(2)}} J = H^\top G + 2\lambda W^{(2)}$
3. $\nabla_H J = G W^{(2)\top}$
4. $G' \leftarrow \nabla_H J \odot \mathbf{1}[U^{(1)} > 0]$ (ReLU gate)
5. $\nabla_{W^{(1)}} J = X^\top G' + 2\lambda W^{(1)}$

Update (SGD):

$$W^{(k)} \leftarrow W^{(k)} - \epsilon \nabla_{W^{(k)}} J$$

Complexity: $O(w)$ for both passes, where $w =$ number of weights.

Module 3 Summary: Why Deep Learning?

- Linear models cannot capture feature interactions. Deep learning learns the transformation $\phi(x;\theta)$ from data.
- Curse of dimensionality: configurations grow exponentially with dimension. High-dimensional spaces are mostly empty.
- Smoothness alone is insufficient. Local methods (kNN, kernels, trees) require $O(k)$ examples for $O(k)$ regions.
- Deep networks enable non-local generalisation through shared features.
- Manifold hypothesis: data lies near low-dimensional manifolds. Deep learning learns manifold coordinates.
- XOR shows representation learning: one hidden layer makes XOR linearly separable.

Module 4 Summary: Neural Networks

- Feedforward networks compose layers: $f(x) = f^{(L)}(\cdots f^{(1)}(x))$. Hidden layers are not directly supervised.
- Cost functions are negative log-likelihoods. Gaussian gives MSE, Bernoulli and categorical give cross-entropy.
- Negative log-likelihood improves gradients by avoiding saturation in sigmoid and softmax.
- ReLU is standard due to simplicity, non-saturation for active units, and efficiency. Leaky variants reduce dead units.
- Universal approximation: shallow networks can approximate any function but may need exponential width.
- Depth improves efficiency by representing hierarchical structure with compositional functions.

Module 5 Summary: Learning Through Backpropagation

- SGD replaces full gradients with minibatches. This reduces cost and enables large-scale training.
- ML pipeline: dataset, model, cost function, and optimization define any learning algorithm.
- Computational graphs define variables and operations as a DAG for automatic differentiation.
- Chain rule: gradients propagate through Jacobians in reverse order.
- Backpropagation is dynamic programming over the graph. Cost is proportional to graph size.
- Backprop computes gradients. SGD updates parameters using them.

Training loop: forward pass → loss → backprop → SGD update → repeat

Questions?

Contact Information	
Author	Emmanuel Djegou, PhD
Topic Area	Data Science, Statistical Modeling & Survival Analysis
Email	emmanueldjegou5@gmail.com
LinkedIn	linkedin.com/in/emmanuel-djegou-phd-5652b2254